Approximate Dynamic Programming for Stochastic Airport Operations: A Study of Automatic State Representation Methods

Jeroen Fränzel

February 9, 2026

Abstract

This thesis investigates automatic state representation methods for Approximate Dynamic Programming (ADP) applied to stochastic airport gate assignment problems. We formulate the gate assignment problem as a finite-horizon Markov Decision Process (MDP) with stochastic arrivals and service times, then generate state-transition trajectories via simulation under baseline policies. We study three approaches to basis function construction for linear value function approximation: handcrafted domain features, Proto-Value Functions derived from spectral analysis of the state-transition graph, and successor feature representations. Our analysis focuses on approximation quality, interpretability, and scalability under varying congestion and disruption scenarios. This work contributes a rigorous comparison of representation learning methods in a safety-critical operational domain where interpretability and mathematical transparency are essential.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Approach . . . 3
- 1.4 Contributions . . . 4
- 1.5 Thesis Outline . . . 4

2 Background **5**
- 2.1 Markov Decision Processes . . . 5
- 2.2 Approximate Dynamic Programming . . . 5
  - 2.2.1 Linear Value Function Approximation . . . 5
  - 2.2.2 Temporal Difference Learning . . . 6
- 2.3 Proto-Value Functions . . . 6
  - 2.3.1 Graph Laplacian . . . 6
  - 2.3.2 Spectral Basis Functions . . . 6
  - 2.3.3 Interpretation . . . 6
- 2.4 Successor Features . . . 6
- 2.5 Related Work . . . 7

3 MDP Formulation of Gate Assignment **8**
- 3.1 Problem Description . . . 8
- 3.2 State Space . . . 8
- 3.3 Action Space . . . 8
- 3.4 Transition Dynamics . . . 8
  - 3.4.1 Deterministic Time Evolution . . . 9
  - 3.4.2 Gate Occupancy Updates . . . 9
  - 3.4.3 Stochastic Arrivals . . . 9
  - 3.4.4 Queue Updates . . . 9
- 3.5 Reward Function . . . 9
- 3.6 Assumptions . . . 9
- 3.7 Complexity Analysis . . . 9

4 Trajectory Generation via Simulation **10**

5 State Representation Methods **11**

6 Approximate Dynamic Programming Implementation **12**

7 Experimental Design **13**

8 Results and Analysis **14**

9 Conclusion and Future Work **15**

A Mathematical Proofs **17**

B Implementation Details **18**

Chapter 1

Introduction

1.1 Motivation

Airport operations face increasing complexity due to growing air traffic volumes, resource constraints, and operational uncertainties. Gate assignment—the problem of allocating arriving aircraft to terminal gates—is a critical scheduling task that affects passenger experience, airline efficiency, and airport throughput.

Traditional approaches to gate assignment often rely on deterministic optimization or heuristic rules. However, real-world operations involve significant stochastic elements: uncertain arrival times due to weather and air traffic control delays, variable turnaround service times, and unpredictable disruptions. These uncertainties motivate the use of stochastic sequential decision-making frameworks.

1.2 Problem Statement

We address the following research question:

> *How can we automatically construct effective state representations for Approximate Dynamic Programming in stochastic airport gate assignment, and how do learned representations compare to handcrafted domain features in terms of approximation quality, interpretability, and scalability?*

1.3 Approach

Our methodology consists of the following stages:

1. **MDP Formulation**: Define the gate assignment problem as a discrete-time, finite-horizon MDP with explicit state, action, transition, and reward components.

2. **Trajectory Generation**: Simulate episodes under simple baseline policies (random assignment, greedy assignment) to collect state-transition data.

3. **Graph Construction**: Build a weighted state-transition graph from observed trajectories, where nodes represent states and edge weights reflect empirical transition probabilities.

4. **Representation Learning**: Construct basis functions via:
   - Proto-Value Functions (PVFs): eigenvectors of the graph Laplacian
   - Successor Features: expected discounted state occupancy
   - Handcrafted Features: domain-specific engineered features (baseline)
5. **ADP with Linear VFA**: Apply temporal difference learning to estimate value functions using each representation, compare performance.
6. **Evaluation**: Test learned policies under congestion and disruption scenarios, analyze approximation quality and robustness.

1.4 Contributions

- A mathematically rigorous MDP formulation of stochastic gate assignment suitable for simulation and analysis
- Empirical comparison of automatic vs. handcrafted state representations in a safety-critical operational domain
- Analysis of interpretability and scalability trade-offs in representation learning for ADP
- Open-source implementation enabling reproducibility and extension

1.5 Thesis Outline

The remainder of this thesis is organized as follows:

- **Chapter 2**: Background on MDPs, ADP, and spectral methods
- **Chapter 3**: Formal MDP formulation of gate assignment
- **Chapter 4**: Trajectory generation methodology
- **Chapter 5**: State representation methods
- **Chapter 6**: ADP implementation with linear VFA
- **Chapter 7**: Experimental design
- **Chapter 8**: Results and analysis
- **Chapter 9**: Conclusions and future work

Chapter 2

Background

2.1 Markov Decision Processes

Definition 2.1 (Finite-Horizon MDP)**.** A finite-horizon Markov Decision Process is a tuple $(\mathcal{S}, \mathcal{A}, P, R, T)$ where:

- $\mathcal{S}$ is a finite state space
- $\mathcal{A}$ is a finite action space
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition probability function
- $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function
- $T \in \mathbb{N}$ is the time horizon

A policy $\pi : \mathcal{S} \times \{0, \ldots, T-1\} \to \mathcal{A}$ maps states and time to actions. The value function under policy π is:

$$V_t^\pi(s) = \mathbb{E}^\pi \left[\sum_{k=t}^{T-1} R(s_k, a_k) \mid s_t = s \right] \tag{2.1}$$

The optimal value function satisfies the Bellman optimality equation:

$$V_t^*(s) = \max_{a \in \mathcal{A}} \left\{ R(s, a) + \sum_{s' \in \mathcal{S}} P(s'|s, a) V_{t+1}^*(s') \right\} \tag{2.2}$$

2.2 Approximate Dynamic Programming

When the state space is large, exact dynamic programming becomes computationally intractable. Approximate Dynamic Programming addresses this through value function approximation.

2.2.1 Linear Value Function Approximation

We approximate the value function as a linear combination of basis functions:

$$\hat{V}(s; \theta) = \sum_{i=1}^{k} \theta_i \phi_i(s) = \phi(s)^\top \theta \tag{2.3}$$

where $\phi : \mathcal{S} \to \mathbb{R}^k$ maps states to feature vectors and $\theta \in \mathbb{R}^k$ are learnable weights.

2.2.2 Temporal Difference Learning

The TD(0) update for linear VFA is:

$$\theta \leftarrow \theta + \alpha \left[r + \hat{V}(s'; \theta) - \hat{V}(s; \theta) \right] \phi(s) \tag{2.4}$$

where $\alpha > 0$ is the learning rate.

2.3 Proto-Value Functions

Proto-Value Functions [Mahadevan and Maggioni, 2007] provide a task-independent basis for value function approximation derived from the topology of the state space.

2.3.1 Graph Laplacian

Given a weighted graph $G = (V, E, W)$ representing state transitions:

$$L = D - W \tag{2.5}$$

where W is the weighted adjacency matrix and D is the degree matrix with $D_{ii} = \sum_j W_{ij}$.

The normalized Laplacian is:

$$\mathcal{L} = I - D^{-1/2} W D^{-1/2} \tag{2.6}$$

2.3.2 Spectral Basis Functions

The eigenvectors of $\mathcal{L}$ corresponding to the smallest eigenvalues form a smooth basis over the state space. Let $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_{|\mathcal{S}|}$ be the eigenvalues with eigenvectors $v_1, \ldots, v_{|\mathcal{S}|}$.

The first k non-trivial eigenvectors $\{v_2, \ldots, v_{k+1}\}$ serve as PVFs:

$$\phi_i(s) = v_{i+1}(s), \quad i = 1, \ldots, k \tag{2.7}$$

2.3.3 Interpretation

PVFs capture the "geometry" of the state space:

- Lower eigenvalues correspond to smoother functions over the graph
- They identify bottlenecks and clusters in state-transition structure
- Task-independent: derived from dynamics, not rewards

2.4 Successor Features

Successor features [Dayan, 1993, Barreto et al., 2017] represent states by their expected future occupancy.

Definition 2.2 (Successor Representation)**.** The successor representation under policy π with discount factor γ is:

$$\psi^\pi(s) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \mathbb{I}[s_t = s] \mid s_0 = s \right] \tag{2.8}$$

Successor features generalize this to feature expectations:

$$\psi^\pi(s) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \phi(s_t) \mid s_0 = s \right] \tag{2.9}$$

These can be learned via TD updates and enable transfer learning across tasks with different reward functions.

2.5 Related Work

[Summary of relevant literature on airport scheduling, ADP applications in operations research, spectral methods in RL, etc.]

Chapter 3

MDP Formulation of Gate Assignment

3.1 Problem Description

An airport has G gates available for aircraft turnaround operations. Flights arrive stochastically over a finite time horizon T. Each flight requires assignment to a gate for a random service duration (disembarkation, cleaning, refueling, boarding). The objective is to minimize waiting times and maximize throughput while respecting operational constraints.

3.2 State Space

The state at time t is a tuple $s_t = (t, \mathbf{g}_t, \mathbf{q}_t)$ where:

- $t \in \{0, 1, \ldots, T\}$: current time index
- $\mathbf{g}_t \in \{0, 1, \ldots, S_{\max}\}^G$: gate occupancy vector
  - $g_i = 0$: gate i is available
  - $g_i = s > 0$: gate i occupied, s time steps remaining
- $\mathbf{q}_t$: queue of waiting flights
  - Each flight characterized by: arrival time, gate type compatibility, priority

3.3 Action Space

At each time step, the decision-maker selects an action $a_t \in \mathcal{A}_t$ where:

$$\mathcal{A}_t = \{(f, g) : f \in \mathbf{q}_t, g \in \text{Available}(\mathbf{g}_t), \text{Compatible}(f, g)\} \cup \{\text{wait}\} \tag{3.1}$$

An action (f, g) assigns waiting flight f to available gate g. The "wait" action defers assignment.

3.4 Transition Dynamics

The transition from s_t to s_{t+1} involves:

3.4.1 Deterministic Time Evolution

$$t \to t + 1 \tag{3.2}$$

3.4.2 Gate Occupancy Updates

For each gate i:

$$g_i^{t+1} = \begin{cases} s - 1 & \text{if } g_i^t = s > 1 \\ 0 & \text{if } g_i^t = 1 \text{ (service completes)} \\ \text{Service}(f) & \text{if action assigns flight } f \text{ to gate } i \end{cases} \tag{3.3}$$

where $\text{Service}(f) \sim P_{\text{service}}$ is a random service duration.

3.4.3 Stochastic Arrivals

New flights arrive according to a Poisson or scheduled process:

$$\text{NewArrivals}_{t+1} \sim P_{\text{arrival}}(t) \tag{3.4}$$

3.4.4 Queue Updates

$$\mathbf{q}_{t+1} = (\mathbf{q}_t \setminus \{f\}) \cup \text{NewArrivals}_{t+1} \tag{3.5}$$

where f is the assigned flight (if any).

3.5 Reward Function

The immediate reward penalizes waiting and encourages throughput:

$$R(s_t, a_t) = -c_{\text{wait}} \cdot |\mathbf{q}_t| - c_{\text{overflow}} \cdot \mathbb{I}[|\mathbf{q}_t| > Q_{\max}] + c_{\text{assign}} \cdot \mathbb{I}[\text{assignment made}] \tag{3.6}$$

3.6 Assumptions

Assumption 3.1. Service times are independent and identically distributed random variables with finite support.

Assumption 3.2. Arrival processes are stationary within each time period.

Assumption 3.3. Gates are homogeneous (can be relaxed to gate types: domestic/international, wide-body/narrow-body).

3.7 Complexity Analysis

The state space size is:

$$|\mathcal{S}| \approx T \cdot (S_{\max} + 1)^G \cdot \binom{F_{\max} + Q_{\max}}{Q_{\max}} \tag{3.7}$$

where $F_{\max}$ is the maximum number of flight types and $Q_{\max}$ is the queue capacity.

For realistic parameters ($G = 10$, $S_{\max} = 10$, $T = 100$, $Q_{\max} = 20$), exact dynamic programming is intractable, motivating the need for approximation.

Chapter 4

Trajectory Generation via Simulation

[To be developed: baseline policies, simulation algorithm, data collection]

Chapter 5

State Representation Methods

[To be developed: PVF computation, successor features, handcrafted features]

Chapter 6

Approximate Dynamic Programming Implementation

[To be developed: TD learning with linear VFA, convergence properties]

Chapter 7

Experimental Design

[To be developed: parameter settings, evaluation metrics, scenarios]

Chapter 8

Results and Analysis

[To be developed: comparative performance, interpretation, scalability]

Chapter 9

Conclusion and Future Work

[To be developed: summary of contributions, limitations, extensions]

Bibliography

André Barreto, Will Dabney, Rémi Munos, Jonathan J Hunt, Tom Schaul, Hado P van Hasselt, and David Silver. Successor features for transfer in reinforcement learning. *Advances in neural information processing systems*, 30, 2017.

Peter Dayan. Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4):613–624, 1993.

Sridhar Mahadevan and Mauro Maggioni. Proto-value functions: A laplacian framework for learning representation and control in markov decision processes. *Journal of Machine Learning Research*, 8:2169–2231, 2007.

Appendix A

Mathematical Proofs

[To be developed: detailed derivations, convergence proofs]

Appendix B

Implementation Details

[To be developed: algorithms in pseudocode, computational considerations]